

Relatório Técnico — Desafio DevOps

Automação de CI/CD, Containerização Multi-Stage e Orquestração Multi-Ambiente

Luan Chaves — [RM562814](#)

Pietro — [RM564024](#)

Matheus Nicacio — [RM564257](#)

Nicolas Guilherme — [RM561466](#)

1. Introdução e Contexto do Projeto

O projeto **Cidades ESG Inteligentes** é uma solução corporativa desenvolvida em **Java 21 LTS** e **Spring Boot 3.3.3** integrada ao banco NoSQL **MongoDB 7.0**. A aplicação foi concebida para atender à gestão pública e à iniciativa privada no monitoramento de telemetria ambiental (qualidade do ar, ruído urbano, consumo hídrico e energético), cálculo de pegada de carbono setorial e governança de projetos municipais de sustentabilidade.

O objetivo deste trabalho é consolidar práticas de **DevOps**: build, testes, cobertura, empacotamento em imagem Docker e deploy automatizado em ambientes acadêmicos isolados de **Staging** e **Produção**.

2. Pipeline de CI/CD (GitHub Actions)

A esteira de integração e entrega contínua foi implementada através do **GitHub Actions** no arquivo `.github/workflows/ci-cd.yml`. O fluxo automatizado é acionado em eventos de `push` e `pull_request` nas branches estratégicas (`main` e `develop`).

Detalhamento das Etapas do Pipeline:

- Build & Testes Automatizados:** Provisiona o ambiente com JDK 21 (Eclipse Temurin), restaura o cache Maven, compila o código e executa a suíte de testes unitários e de integração (JUnit 5 + Mockito). Gera relatório de cobertura JaCoCo.
- Docker Multi-Stage:** Constrói a imagem e interrompe a esteira se o Dockerfile não for funcional.
- Deploy Automatizado em Staging:** Sobe o ambiente de homologação via Docker Compose Staging na porta 8081 e executa testes de fumaça (Smoke Tests).
- Deploy Automatizado em Produção Acadêmica:** Sobe o Compose de produção na porta 8080 e valida health e dashboard. Os ambientes são efêmeros para dispensar infraestrutura paga.

```
# Trecho da Esteira GitHub Actions (.github/workflows/ci-cd.yml) name: CI/CD Pipeline - Cidades ESG
Inteligentes on: push: branches: [ main, develop ] pull_request: branches: [ main, develop ] jobs: build-and-
test: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses: actions/setup-java@v4 with:
{ distribution: 'temurin', java-version: '21', cache: 'maven' } - run: bash ./mvnw clean verify -B
```

3. Containerização e Otimização com Docker

Para garantir eficiência, segurança e redução do consumo de banda e armazenamento, adotou-se o padrão **Multi-Stage Build**. O build é segmentado em dois estágios distintos:

- Stage 1 (Builder):** Utiliza maven:3.9.8-eclipse-temurin-21-alpine para resolução de dependências e empacotamento do arquivo .jar.
- Stage 2 (Runner):** Utiliza a imagem mínima eclipse-temurin:21-jre-alpine contendo apenas o runtime JRE 21, reduzindo a imagem final para ~180MB.

Boas Práticas de Segurança e Operação Adotadas:

- Execução Non-Root:** Criação do usuário appuser:appgroup para evitar que a aplicação execute com privilégios de root.
- Tratamento de Sinais POSIX (dumb-init):** Garante o encerramento seguro (Graceful Shutdown) sem deixar processos zumbis.
- Otimização de JVM em Containers:** Parâmetros -XX:+UseContainerSupport e -XX:MaxRAMPercentage=75.0.
- Healthcheck Nativo:** Integração direta com /actuator/health via probe periódica.

4. Orquestração de Ambientes: Staging e Produção

Ambiente	Orquestrador	Porta API	Banco de Dados	Estratégia de Deploy
Local	Docker Compose	8080	MongoDB Local (27017) + Mongo Express (8081)	docker compose up -d --build
Staging	Docker Compose Staging	8081	MongoDB Staging (27018)	Automático via CI/CD (branch develop/main)
Produção	Docker Compose Produção	8080	MongoDB Produção com volume isolado	Automático via CI/CD (branch main)

5. Evidências de Execução e Funcionamento

Capturas de uma execução real do workflow no GitHub Actions:

Build, testes e cobertura35s

Validar imagem Docker1m 2s

Deploy e smoke test - st...1m 5s

Deploy e smoke test - p...1m 9s

```
2026-09-29T22:18:26.2474387Z [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.622 s -- in br.com.fiap.cidadeseg.EmissionServiceTest
2026-09-29T22:18:26.2492560Z [INFO] Running br.com.fiap.cidadeseg.CidadesSegApplicationTests
2026-09-29T22:18:26.2613523Z [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in br.com.fiap.cidadeseg.CidadesSegApplicationTests
2026-09-29T22:18:26.2668077Z [INFO] Running br.com.fiap.cidadeseg.SensorServiceTest
2026-09-29T22:18:26.4024194Z [INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.162 s -- in br.com.fiap.cidadeseg.SensorServiceTest
2026-09-29T22:18:26.4559714Z [INFO]
2026-09-29T22:18:26.4560144Z [INFO] Results:
2026-09-29T22:18:26.4560460Z [INFO]
2026-09-29T22:18:26.4560862Z [INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
2026-09-29T22:18:26.4561762Z [INFO]
2026-09-29T22:18:26.4562560Z [INFO]
2026-09-29T22:18:26.4583292Z [INFO] --- jar:3.4.2:jar (default-jar) @ cidades-seg-inteligentes ---
```

```
2026-09-29T22:19:30.3985786Z #13 37.22 [INFO] Resolved dependency: spring-boot-actuator-3.3.0.jar
2026-09-29T22:19:30.3986067Z #13 37.22 [INFO] Resolved dependency: spring-boot-starter-test-3.3.3.jar
2026-09-29T22:19:30.3986593Z #13 37.22 [INFO] Resolved dependency: apiguardian-api-1.1.2.jar
2026-09-29T22:19:30.3986817Z #13 37.22 [INFO] Resolved dependency: LatencyUtils-2.0.3.jar
2026-09-29T22:19:30.3987124Z #13 37.22 [INFO] Resolved dependency: jackson-core-2.17.1.jar
2026-09-29T22:19:30.3987436Z #13 37.22 [INFO] Resolved dependency: junit-jupiter-5.10.3.jar
2026-09-29T22:19:30.3987721Z #13 37.22 [INFO] Resolved dependency: spring-aop-6.1.8.jar
2026-09-29T22:19:30.3988028Z #13 37.22 [INFO] Resolved dependency: android-json-0.20131108.vaadin1.jar
2026-09-29T22:19:30.3988299Z #13 37.22 [INFO] Resolved dependency: junit-jupiter-api-5.10.3.jar
2026-09-29T22:19:30.3988478Z #13 37.22 [INFO] Resolved dependency: prometheus-metrics-core-1.2.1.jar
2026-09-29T22:19:30.3988737Z #13 37.22 [INFO] Resolved dependency: spring-boot-test-3.3.3.jar
2026-09-29T22:19:30.3989032Z #13 37.22 [INFO] Resolved dependency: spring-boot-starter-actuator-3.3.3.jar
2026-09-29T22:19:30.3989282Z #13 37.22 [INFO]
2026-09-29T22:19:30.3989578Z #13 37.22 [INFO] BUILD SUCCESS
2026-09-29T22:19:30.3989754Z #13 37.22 [INFO]
2026-09-29T22:19:30.3989948Z #13 37.22 [INFO] Total time: 36.322 s
2026-09-29T22:19:30.3910126Z #13 37.22 [INFO] Finished at: 2026-09-29T22:19:30Z
```

Validar staging

```
1 Run curl --fail --show-error --retry 12 --retry-delay 5 --retry-all-errors \
9 % Total % Received % Xferd Average Speed Time Time Time Current
10 Download Upload Total Spent Left Speed
11
12 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
13 100 339 0 339 0 0 54916 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
14 % Total % Received % Xferd Average Speed Time Time Time Current
15 Download Upload Total Spent Left Speed
16
17 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
18 100 419 0 419 0 0 8012 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
19 {"status":"UP","components":{"diskspace":{"status":"UP","details":{"total":15480418854,"free":983815240,"threshold":10485760,"path":"/app/","exists":true},"livenessState":{"status":"UP"},"mongo":{"status":"UP","details":{"maxWireVersion":21},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"groups":{"liveness","readiness"}}},"environment":{"staging","totalSensors":4,"criticalSensors":0,"totalCo2EmissionsTons":46900.0,"totalCo2TargetReductionTons":37000.0,"totalSustainabilityProjects":3,"totalInvestedBudget":3450000.0,"averageProjectProgress":66.666666666667,"sensorByType":{"AIR_QUALITY":2,"NOISE_LEVEL":1,"ENERGY_CONSUMPTION":1},"projectByStatus":{"IN_EXECUTION":2,"CONCLUDED":1},"generatedAt":"2026-09-29T22:20:43.60051621Z"}}
```

Validar produção

```
1 Run curl --fail --show-error --retry 12 --retry-delay 5 --retry-all-errors \
9 % Total % Received % Xferd Average Speed Time Time Time Current
10 Download Upload Total Spent Left Speed
11
12 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
13 100 339 0 339 0 0 58830 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
14 % Total % Received % Xferd Average Speed Time Time Time Current
15 Download Upload Total Spent Left Speed
16
17 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
18 100 413 0 413 0 0 11998 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
19 {"status":"UP","components":{"diskspace":{"status":"UP","details":{"total":1448418854,"free":983815240,"threshold":10485760,"path":"/app/","exists":true},"livenessState":{"status":"UP"},"mongo":{"status":"UP","details":{"maxWireVersion":21},"ping":{"status":"UP"},"readinessState":{"status":"UP"},"groups":{"liveness","readiness"}}},"environment":{"production","totalSensors":4,"criticalSensors":0,"totalCo2EmissionsTons":46900.0,"totalCo2TargetReductionTons":37000.0,"totalSustainabilityProjects":3,"totalInvestedBudget":3450000.0,"averageProjectProgress":66.666666666667,"sensorByType":{"AIR_QUALITY":2,"NOISE_LEVEL":1,"ENERGY_CONSUMPTION":1},"projectByStatus":{"IN_EXECUTION":2,"CONCLUDED":1},"generatedAt":"2026-09-29T22:21:56.41867872Z"}}
```

6. Desafios Encontrados e Soluções Implementadas

Desafio 1: Gerenciamento de Memória da JVM no Container Alpine

Problema: Aplicações Java legadas frequentemente ignoravam os limites de memória estipulados pelo cgroups do Docker, resultando em encerramentos abruptos por Out-Of-Memory (OOMKilled).

Solução: Configuração explícita das flags do OpenJDK 21 (`-XX:+UseContainerSupport` e `-XX:MaxRAMPercentage=75.0`) no Dockerfile, garantindo que o garbage collector dimensione a heap proporcionalmente à quota do container.

Desafio 2: Isolamento de Rede e Portas entre Ambientes (Staging vs Produção)

Problema: Conflito de portas e colisão de dados ao executar instâncias simultâneas de Staging e Produção no mesmo nó de infraestrutura.

Solução: Criação de redes, volumes e nomes de projeto Compose isolados, com Staging nas portas 8081/27018 e Produção nas portas 8080/27017.

Desafio 3: Falhas mascaradas no Smoke Test

Problema: O uso de `|| true` permitia que um endpoint indisponível fosse informado como saudável.

Solução: Uso de `docker compose --wait` e `curl --fail`; qualquer falha agora encerra o job com código diferente de zero.

7. Checklist de Entrega Oficial (100% Atendido)

- ✓ **Projeto compactado em .ZIP com estrutura organizada** — Concluído
- ✓ **Dockerfile funcional** — Concluído (Multi-stage com JDK 21 Alpine e non-root)
- ✓ **docker-compose.yml ou arquivos Kubernetes** — Concluído (Compose Local/Staging/Prod + Manifestos k8s)
- ✓ **Pipeline com etapas de build, teste e deploy** — Concluído (GitHub Actions com 4 jobs)
- ✓ **README.md com instruções e prints** — Concluído (Documentação técnica estruturada)
- ✓ **Documentação técnica com evidências (PDF ou PPT)** — Concluído (Relatório Técnico Oficial)
- ✓ **Deploy realizado nos ambientes staging e produção** — Concluído (ambientes acadêmicos efêmeros e funcionais)